

Rreport of my coursework

About Scotland Yard

—Ziyang Wang
(rl24988)

Basically, this report is to reflex my problems, the disadvantages and advantages in my work.

As a beginning, I need to report what we done in coursework. Simply, we finish all the test in cw-model. That work is aiming to implement the full version of the game.

Back to the topic, I wil talk about my problems in this coursework. First of all, I am not very familiar of java and honestly this work takes me a lot of time. There are still amount of ways to make my code simple and readable. For example, some for-loop can be replace by stream operation. Because of the first time to finish such huge task, I can not realise which method should I pick to slove problems easily immediately. Secondly, I can not use git easily at first that makes me feel difficult to cooperate with my teammates.

When it comes to my advantages, I will gald to admit that I have finished lots of test in cw-model task. Obviously, I make a huge contribution to my team. Esepccially, there are two test which confused my team for a long time, testGamenotoverIfMrXcorneredButstillescape and testGamenotoverIfsomedetectivestuck. The reason that we come over is because I find a crucial bug in our advance method and I fixed it personally without doubt.

```
public final class MyGameStateFactory implements Factory<GameState> { 2 usages
    private final class MyGameState implements GameState { 2 usages
        public GameState advance(Move move) {

            // when detective's turn comes, check if detective can move
            Set<Piece> detectivesWithMoves = new HashSet<>();
            for (Player detective : newDetectives) {
                ImmutableSet<Move> movesForDetective = calculateAvailableMoves(setup, newDetectives, detective);
                if (!movesForDetective.isEmpty()) {
                    detectivesWithMoves.add(detective.piece());
                }
            }
            if (!detectivesWithMoves.isEmpty()) {
                newRemaining = ImmutableSet.copyOf(detectivesWithMoves);
            } else {
                newRemaining = ImmutableSet.of(mrX.piece());
            }
        } else {
            // after moving, remove player from remaining
            newRemaining = remaining.stream()
                .filter( Piece p -> !p.equals(mover))
                .collect(ImmutableSet.toImmutableSet());
            if (newRemaining.isEmpty()) {
                newRemaining = ImmutableSet.of(mrX.piece());
            }
        }
    }
}
```

Like the screenshot above, our team forgot to check whether detective can move or not when it is detective's turn which makes us fail the test again and again. We even rewrite our single and doble move method. After I point out this problem, I add those codes so that we can come over the test we stuck.